DATABRICKS COMMUNITY EDITION

Mini Project

E-Commerce Sales Analytics

End-to-End PySpark Pipeline with Advanced Analytics

Platform: Databricks Community Edition | Language: PySpark (Python)

1. Project Overview

1.1 Problem Statement

Real-world e-commerce platforms generate enormous volumes of transactional data across orders, products, customers, and logistics. The challenge is not just storing this data, but building a robust distributed processing pipeline capable of handling scale, ensuring data quality, and surfacing actionable business intelligence.

This project simulates that environment using realistic synthetic datasets, and guides you through building a production-inspired PySpark pipeline on Databricks Community Edition — covering everything from raw ingestion to optimized analytics-ready tables.

1.2 Project Objectives

- Design a multi-layer data architecture (Bronze, Silver, Gold) following Lakehouse principles
- Ingest and validate raw CSV/JSON data with schema enforcement and null-handling strategies
- Apply complex transformations using DataFrame API, Spark SQL, and window functions
- Implement partitioning, caching, and broadcast joins to optimize query performance
- Build aggregated analytical datasets for revenue, product, and customer reporting
- Derive business KPIs including CLV (Customer Lifetime Value), churn signals, and cohort retention
- Monitor Spark execution plans and tune query performance using explain() and Spark UI

1.3 Why This Project Demonstrates Real Experience

This project is deliberately structured around decisions a working PySpark engineer makes daily: choosing between narrow and wide transformations, deciding when to persist vs. recompute, writing idempotent pipelines, and understanding the trade-offs between broadcast joins and sort-merge joins. The business logic layer also mimics actual BI team requirements — not toy aggregations.

Tech Stack

- Databricks Community Edition (DBR 13.x / Spark 3.4+)
- PySpark DataFrame API + Spark SQL
- Delta Lake (available in Databricks Community)
- Python 3.10 — used for utility functions and schema definitions
- Databricks Notebooks with %md, %sql, and %python magic commands
- DBFS (Databricks File System) for dataset storage

2. Architecture & Data Model

2.1 Medallion / Lakehouse Architecture

The pipeline follows the Medallion architecture — a layered data design pattern that separates raw data, cleansed data, and business-ready aggregations. This structure is used in production Databricks and Azure Data Factory pipelines worldwide.

Layer	Purpose & Description
Bronze (Raw)	Raw ingestion layer. Data lands exactly as received from source files (CSV/JSON). No transformations, no drops. Append-only. Schema is inferred with options to handle malformed records.
Silver (Cleansed)	Type-cast, validated, deduplicated, and enriched data. Joins between domain entities (orders + customers + products) happen here. Business-logic columns are derived (e.g., order_total, days_to_ship).
Gold (Aggregated)	Aggregated, analytics-ready tables. These are the final outputs consumed by dashboards and reporting. Partitioned by date for efficient querying. Optimized for read performance.

2.2 Dataset Schema

You will work with four synthetic datasets. Together they form a realistic relational model:

Dataset	Key Fields & Notes
orders.csv	order_id, customer_id, product_id, quantity, order_date, shipped_date, status, payment_method, discount_pct — ~100K rows
customers.csv	customer_id, name, email, city, state, country, signup_date, segment (B2B/B2C) — ~20K rows
products.csv	product_id, product_name, category, subcategory, unit_price, cost_price, supplier_id — ~2K rows
returns.csv	return_id, order_id, return_date, reason_code, refund_amount — ~8K rows

2.3 Data Generation Strategy

Since Community Edition has no live data source connectors, you will generate all datasets programmatically using Python's Faker library and random module inside a Databricks notebook. This approach is important: it lets you control data distribution, inject known anomalies (nulls, duplicates, late shipments), and test your pipeline against edge cases.

- Generate data with realistic distributions — Pareto-distributed sales (top 20% products drive 80% revenue)
- Inject 3-5% NULL values in non-key columns to simulate dirty data
- Include ~2% duplicate order rows to test deduplication logic
- Generate customers across 3 time-based cohorts (2021, 2022, 2023) for cohort analysis
- Save generated files to DBFS: /FileStore/ecommerce/raw/

3. Implementation Guide

The following phases walk through the complete implementation. Each phase is a separate Databricks notebook. This structure keeps the pipeline modular and easier to debug — a standard practice in production environments.

Phase 1: Environment Setup & Data Generation

- Create a new Databricks cluster (DBR 13.x LTS, single-node is fine for Community Edition)
- Create a top-level folder: /ecommerce-pipeline/ in your Workspace
- Notebook: 00_data_generation — install Faker via %pip install faker, write generator functions for each of the four datasets
- Save all CSVs to DBFS path: /FileStore/ecommerce/raw/<dataset>.csv
- Verify file creation using dbutils.fs.ls('/FileStore/ecommerce/raw/')
- Key decision: Write generator functions that accept a seed parameter for reproducibility

Phase 2: Bronze Layer — Raw Ingestion

- Notebook: 01_bronze_ingestion
- Define explicit StructType schemas for each dataset using StructField — never rely on inferSchema=True in production (it triggers an extra full scan)
- Read CSVs with spark.read.csv() using options: header=True, mode='PERMISSIVE', columnNameOfCorruptRecord='_corrupt_record'
- Write to Delta format at /FileStore/ecommerce/bronze/<table> using .write.format('delta').mode('overwrite')
- Register tables in the Hive Metastore using spark.sql('CREATE TABLE IF NOT EXISTS bronze_orders USING DELTA LOCATION ...')
- Log row counts and schema before/after write — build a simple audit dictionary in Python
- Concept focus: Understand the difference between PERMISSIVE, DROPMALFORMED, and FAILFAST read modes

Phase 3: Silver Layer — Cleansing & Transformation

- Notebook: 02_silver_transform — this is the most complex notebook in the pipeline
- Step 1 — Schema validation: Cast all columns to correct types, handle date parsing with to_date() and handle format mismatches
- Step 2 — Null handling: Use conditional strategy — fill numeric nulls with median (use approxQuantile), drop rows where order_id or customer_id is null (non-nullable keys)
- Step 3 — Deduplication: Use dropDuplicates(['order_id']) after ordering by a timestamp to keep the latest record (use Window + row_number)

- Step 4 — Derived columns: Calculate order_revenue = quantity * unit_price * (1 - discount_pct), days_to_ship = datediff(shipped_date, order_date), is_returned = join with returns table
- Step 5 — Join strategy: Join orders with customers and products. Use broadcast join for products (small table) — wrap with broadcast() hint; use SortMergeJoin for the orders-customers join
- Step 6 — Write partitioned Delta: .partitionBy('order_year', 'order_month') for efficient downstream filtering
- Concept focus: Use .explain(True) on your joined DataFrame and read the Physical Plan to identify any unexpected CartesianProduct or BroadcastNestedLoopJoin

Phase 4: Gold Layer — Aggregated Analytics

- Notebook: 03_gold_aggregations — create three Gold tables
- Gold Table 1 — Daily Sales Summary: Group by order_date, category; aggregate total_revenue, total_orders, avg_order_value, total_units_sold, return_rate
- Gold Table 2 — Customer Analytics: Use Window functions to compute: running_total_spend per customer, purchase_rank (rank() by total_spend), days_since_last_order (lag() pattern), customer_lifetime_value estimate
- Gold Table 3 — Product Performance: Category-level metrics with percent_of_total_revenue (using sum over Window), month-over-month revenue change using lag(), top-N products per category using dense_rank() over partition
- Add a churn signal column: flag customers with no orders in 90+ days as at_risk
- Persist Gold tables with .write.format('delta').mode('overwrite').option('overwriteSchema','true')

Phase 5: Advanced Analytics — Window Functions & Cohort Analysis

- Notebook: 04_advanced_analytics
- Cohort Analysis: Assign each customer a cohort based on their first order's month; then calculate retention_rate per cohort per month using a pivot — this produces the classic cohort retention matrix
- Rolling Metrics: Implement 7-day and 30-day rolling revenue using rangeBetween() window frames — understand the difference between rowsBetween and rangeBetween
- ABC Analysis: Classify products into A/B/C tiers using cumulative revenue share (top 80% = A, next 15% = B, bottom 5% = C) — implement using a cumulative sum Window + conditional logic
- Percentile Segmentation: Use ntile(4) to segment customers into quartiles by CLV — label as Platinum, Gold, Silver, Bronze
- Funnel Analysis: Compute order-to-ship conversion, ship-to-return rate, and payment method breakdown as percentage of orders

Phase 6: Performance Optimization

- Notebook: 05_optimization

- Caching strategy: Use .cache() on the Silver orders DataFrame that is reused across Gold tables; call .count() immediately after to trigger materialization; unpersist after Gold writes complete
- Partition tuning: Check spark.conf.get('spark.sql.shuffle.partitions') — default is 200, which is too high for Community Edition. Set to 8-16 based on data size and single-node cluster
- Skew handling: Check for data skew by running a .groupBy('category').count() and comparing partition sizes via rdd.glom().map(len).collect()
- Explain plan audit: For each major transformation, run .explain('formatted') and verify join types, existence of filters being pushed down, and absence of full shuffles where avoidable
- Delta optimization: Run OPTIMIZE and ZORDER BY (order_date) on Silver and Gold tables to improve query performance for date-range filters
- Concept focus: Understand the difference between narrow transformations (map, filter — no shuffle) vs wide transformations (groupBy, join, distinct — triggers shuffle and stage boundaries)

4. Core PySpark Concepts Demonstrated

4.1 Spark Execution Model

Understanding Spark's execution model is what separates someone who uses PySpark from someone who understands it. This project will give you hands-on exposure to the following:

Concept	Where You Apply It in This Project
Lazy Evaluation	Transformations on DataFrames build a logical plan but execute nothing. Action calls (.write, .count, .collect) trigger the physical plan. You observe this by noting no job in Spark UI until an action is called.
DAG & Stages	The Silver notebook has multiple wide transformations (joins, groupBys). The Spark UI Jobs tab shows stages and task distribution. Narrow transformations stay in the same stage.
Shuffle & Partitions	Every groupBy and join involving large tables causes a shuffle. You set spark.sql.shuffle.partitions and observe its effect on task count and job duration.
Catalyst Optimizer	Running .explain('formatted') shows the Optimized Logical Plan vs the Physical Plan. You observe predicate pushdown (filters moving before joins) and constant folding.
AQE (Adaptive Query Execution)	Enable spark.sql.adaptive.enabled=True and observe Spark automatically coalescing small shuffle partitions and switching join strategies at runtime.

4.2 DataFrame API vs Spark SQL

You will implement every Gold aggregation twice: once using the DataFrame API (chained method calls) and once using Spark SQL (via spark.sql() or %sql magic). This is deliberate — it forces you to understand that both compile to the same logical plan, and helps you choose the right tool for readability in team environments.

4.3 Window Functions Deep Dive

Window functions are among the most powerful and most misunderstood features in PySpark. This project uses six distinct window function patterns:

Function	Business Use Case in This Project
row_number()	Deduplication: keep the most recent of duplicate orders by row_number over order_id ordered by ingestion timestamp
rank() / dense_rank()	Product ranking: top products by revenue within each category; customer ranking by CLV

lag() / lead()	Month-over-month change: compare current month revenue to previous month for trend analysis
sum() over Window	Running total spend per customer ordered by order_date; cumulative revenue share for ABC analysis
ntile(n)	Customer quartile segmentation: divide customer base into 4 equal revenue-contribution buckets
rangeBetween()	Rolling 30-day revenue window using date-based range frame instead of row-count frame

4.4 Delta Lake Features Used

- ACID Transactions: Concurrent writes to Delta tables are safe — Databricks handles write conflicts via optimistic concurrency control
- Time Travel: After overwriting Silver tables, use VERSION AS OF 0 in SQL to query the original Bronze-equivalent state
- Schema Evolution: Use .option('mergeSchema', 'true') when adding new derived columns to an existing Delta table without full rewrite
- OPTIMIZE + ZORDER: Compact small files written by streaming or incremental loads; Z-order by order_date for efficient range scans
- Transaction Log: Inspect _delta_log/ in DBFS to understand how Delta tracks every write operation

5. Business KPIs & Analytical Outputs

5.1 Revenue Dashboard Metrics

The Gold daily_sales table should support answering the following questions efficiently (each should run in under 10 seconds on Community Edition):

KPI	Calculation Logic
Total Revenue (MTD/YTD)	SUM(order_revenue) filtered by date range using partition pruning on order_year, order_month
Average Order Value (AOV)	SUM(order_revenue) / COUNT(DISTINCT order_id) — exclude returned orders
Revenue by Category	GROUP BY category with percentage of total using window SUM()
Day-over-Day Growth %	(today_revenue - yesterday_revenue) / yesterday_revenue * 100 using lag()
Return Rate	COUNT(returned_orders) / COUNT(total_orders) per category per month
Discount Impact	SUM(discount_pct * quantity * unit_price) — total revenue foregone to discounts

5.2 Customer Analytics Metrics

Metric	Description & Implementation Note
Customer Lifetime Value (CLV)	Simplified CLV = avg_order_value * purchase_frequency * avg_customer_lifespan. Compute per customer using aggregation then segment into quartiles with ntile(4).
Churn Signal	Flag: last_order_date < current_date - 90 days. Segment into Active, At Risk, Churned. Use datediff() + when/otherwise for conditional classification.
Cohort Retention	Assign cohort month = first order month. For each subsequent month, calculate % of cohort that placed another order. Use pivot() to build the retention matrix.
Purchase Frequency	COUNT(orders) per customer over their full history. Join with cohort assignment to compare frequency across acquisition cohorts.
High-Value Segment	Top 20% of customers by CLV. Cross-reference with churn signal to identify high-value at-risk customers — the most actionable segment.

5.3 Product Performance Metrics

Metric	Implementation
ABC Classification	Cumulative revenue share using ordered window SUM; CASE WHEN cumshare <= 0.8 THEN 'A' WHEN cumshare <= 0.95 THEN 'B' ELSE 'C' END
Margin Analysis	Gross margin = (unit_price - cost_price) / unit_price. Rank by gross margin within category to identify most/least profitable SKUs.
Stockout Proxy	Orders with quantity > rolling avg quantity by 2 standard deviations — potential demand spikes worth flagging.
Cross-sell Index	Products frequently ordered in the same transaction (same order_id). Use self-join on orders then group by product pair.

6. Performance Optimization Reference

6.1 Configuration Parameters to Tune

Set these at the top of your notebooks using spark.conf.set() and observe their effect using the Spark UI:

Parameter	Recommendation for Community Edition & Rationale
spark.sql.shuffle.partitions	Set to 8 or 16. Default of 200 creates 200 tasks for even small shuffles — wasteful on a single-node cluster with 2-4 cores.
spark.sql.adaptive.enabled	Set to True. AQE dynamically coalesces shuffle partitions at runtime, adapts join strategies, and handles skew — major improvement on real data.
spark.sql.adaptive.coalescePartitions.enabled	Set to True (sub-option of AQE). Merges small post-shuffle partitions automatically.
spark.databricks.delta.optimizeWrite.enabled	Set to True in Databricks. Automatically optimizes file sizes during writes to avoid the small-files problem.
spark.sql.broadcastTimeout	Increase to 600 if broadcast joins timeout on Community Edition — the driver has limited memory.

6.2 Common Bottlenecks & How to Diagnose

1. Skewed join: One partition takes 10x longer than others. Diagnosis: Spark UI > Stage > Task metrics — look for max task duration >> median. Fix: salting the join key or enabling AQE skew handling.
2. Too many small files: Delta write produces hundreds of tiny files. Diagnosis: dbutils.fs.ls() shows file count. Fix: Run OPTIMIZE on the table or enable optimizeWrite.
3. Unnecessary full shuffles: GroupBy without filter pushdown scans full table. Diagnosis: .explain() shows no filter in scan. Fix: Add partition filter to leverage partition pruning.
4. Repeated recomputation: Same Silver DataFrame computed 3 times for 3 Gold tables. Fix: .cache() Silver DF before Gold writes, .unpersist() after.
5. Memory spill to disk: Shuffle spill visible in Spark UI executor metrics. Fix: Reduce shuffle partitions or increase executor memory (limited in Community Edition — primarily reduce partitions).

7. Recommended Notebook Structure

Organize your Databricks Workspace as follows. This mirrors how production repositories are structured in engineering teams:

Notebook / File	Contents & Responsibility
00_data_generation	Data generator functions. Parameterized by n_orders, n_customers, seed. Writes CSVs to DBFS. Run once.
01_bronze_ingestion	Schema definitions (StructType). CSV reads with explicit options. Delta writes. Row count audit. Error record logging.
02_silver_transform	All cleansing logic. Type casting. Null imputation. Deduplication using window. Joins with broadcast hints. Derived column logic. Partitioned Delta write.
03_gold_aggregations	Three Gold table builds. DataFrame API version first, then Spark SQL equivalent. Cache Silver before first Gold build. Unpersist after.
04_advanced_analytics	Cohort matrix. Rolling windows. ABC classification. Quartile segmentation. Each section prefaced with a %md cell explaining the business question being answered.
05_optimization	Configuration setting experiments. .explain() output analysis. Delta OPTIMIZE + ZORDER runs. Partition tuning comparisons with timing.
utils/schema_definitions	Centralized StructType schemas as a Python module — imported in ingestion notebook using %run ./utils/schema_definitions

7.1 Notebook Best Practices to Follow

- Open every notebook with a %md cell: project title, notebook purpose, inputs consumed, outputs produced, last updated date
- Define all configuration constants (file paths, table names, date cutoffs) in a single Config cell at the top — never hardcode paths inline
- Use widget parameters (dbutils.widgets) for any value that might change between runs (e.g., processing date, environment)
- Log row counts before and after each major transformation step to detect silent data loss
- Add .explain() cells in development — comment them out (not delete) before sharing; they document your optimization reasoning
- Write assertions: after Silver dedup, assert that order_id column has no duplicates using df.groupBy('order_id').count().filter('count > 1').count() == 0

8. Discussion Points & Depth Questions

These are the kinds of questions a technical interviewer or senior engineer would ask about a project like this. Having worked through this project, you should be able to discuss all of them:

8.1 Architecture Decisions

Question	What to Discuss
Why Bronze-Silver-Gold instead of direct transformation?	Decouples ingestion failures from transformation failures. Allows re-processing Silver from Bronze without re-pulling source data. Enables independent schema evolution at each layer.
Why Delta Lake over Parquet?	ACID transactions, time travel, schema enforcement, automatic small-file compaction via OPTIMIZE. Parquet has none of these — it is just a file format with no transactional guarantees.
Why partition by year/month?	Analytical queries almost always filter on date ranges. Partitioning lets Spark skip entire directories (partition pruning), reducing scan from full table to only relevant months.
When would you choose streaming over batch?	When SLA for data freshness is under 15 minutes. For this project, daily batch is appropriate — adding Structured Streaming would mean maintaining offset checkpoints and handling late-arriving data, which adds operational complexity not justified by daily refresh needs.

8.2 PySpark Technical Depth

Question	What to Discuss
What is the difference between repartition() and coalesce()?	repartition() does a full shuffle and can increase or decrease partition count. coalesce() avoids a shuffle by merging partitions on the same executor — only for decreasing count. Use coalesce before final write to reduce output file count.
When does a broadcast join make sense, and when does it hurt?	Broadcast joins work when one side fits in the driver/executor memory (typically < 200MB, though configurable). They eliminate the shuffle entirely. They hurt when the small table isn't actually small — the broadcast itself becomes expensive. The threshold is controlled by spark.sql.autoBroadcastJoinThreshold.
Why use approxQuantile instead of computing exact median?	Exact median requires sorting all data — O(n log n) and a full shuffle. approxQuantile uses the Greenwald-Khanna algorithm to approximate quantiles in a single pass with bounded error. For imputing nulls, the approximation error is irrelevant.
What causes a data skew, and how do you handle it?	Skew occurs when data is not uniformly distributed across partition keys — e.g., if 60% of orders come from one city. During joins or groupBys, the task handling that key takes 60% of the total time. Solutions: salting (add random prefix to key, multiply small table

	rows), AQE skew handling (auto-detects and splits skewed tasks), or pre-aggregating before the join.

9. Optional Extensions

Once the core pipeline is complete and working, the following extensions demonstrate additional depth. These are worth implementing if you want the project to represent 4+ months of experience rather than 3:

Extension	What It Adds
Incremental / CDC Pattern	Instead of full overwrite, simulate incremental loads by adding new orders daily. Use Delta merge (MERGE INTO) to upsert changed records. This demonstrates understanding of Change Data Capture patterns.
MLlib: Churn Prediction	Use the customer analytics features (days_since_last_order, purchase_frequency, CLV quartile) as features for a simple logistic regression classifier. Train with spark.ml Pipeline, evaluate with BinaryClassificationEvaluator.
Structured Streaming (Mini)	Set up a mock streaming source using rate source or readStream on a directory. Write a simple streaming aggregation (order count per minute) with a watermark. Demonstrates understanding of event-time vs processing-time.
Data Quality Framework	Build a reusable DQ check module: row count validation, null rate checks per column, referential integrity (all order customer_ids exist in customers), duplicate detection. Log results to a Delta audit table.
Parameterized Pipeline Runner	Create a master 00_run_pipeline notebook that runs all notebooks in sequence using dbutils.notebook.run() with parameters (start_date, end_date). Handle failures with try/except and log status to an audit table.

10. Project Deliverables Checklist

Deliverable	Completion Criterion
6 Databricks Notebooks	All notebooks run end-to-end without errors. Clear %md documentation in each.
4 Raw Dataset Files	Stored in DBFS /FileStore/ecommerce/raw/, each with realistic volume and injected anomalies.
Bronze Delta Tables (4)	Raw data preserved as-is in Delta format. Row count audit logged.
Silver Delta Tables (1 joined)	Cleansed, deduplicated, enriched orders joined with customers and products. Partitioned by year/month.

Gold Delta Tables (3)	daily_sales_summary, customer_analytics, product_performance — all with KPI columns as specified.
Optimization Notebook	Demonstrates .explain() analysis, partition tuning, caching strategy, and Delta OPTIMIZE results.
Advanced Analytics Notebook	Cohort retention matrix, ABC classification, customer quartile segmentation working and documented.
README Markdown Cell	Top-level notebook with architecture diagram (text-based), dataset descriptions, and notebook run order.
